National University of Computer & Emerging Sciences



AI ENGINEERING | ASSIGNMENT 1

Building a Small GPT from Scratch

MiniGPT - A GPT-Style Character Language Model

A 4.77M-parameter decoder-only Transformer trained on Tiny Shakespeare, implemented from first principles, tested, evaluated, ablated, and deployed

TEAM MEMBERS

Adbul Rahman Azam (23k-0061)

Syeda Sara Ali (23k-0070)

Talha Rusman (23k-0065)

Instructor:

Sir Farhan Feroze

Code: *github.com/AbdulRahmanAzam/minigpt-shakespear*
Live Demo: *minigpt-shakespeare.containers.snapdeploy.app*

1. Problem and Scenario

Consider a small organization or research team with a limited, domain-specific text collection. The corpus may be far too small to train a general-purpose large language model, yet it can still contain enough statistical structure for a compact model to learn vocabulary, formatting, local dependencies, and stylistic patterns. This motivates the central question of this project:

Can a compact decoder-only Transformer, implemented without prebuilt Transformer layers or pretrained weights, learn the next-character structure of Shakespearean text well enough to generate recognizable language?

MiniGPT answers this question by implementing the entire causal language-model pipeline from first principles in PyTorch. The project includes deterministic character tokenization, scaled dot-product attention, explicit multi-head splitting and merging, sinusoidal positional encoding, pre-normalized Transformer blocks, causal masking, AdamW training, autoregressive sampling, controlled experiments, baseline comparisons, automated tests, a command-line interface, a FastAPI service, and Docker-based deployment.

The complete pipeline is:

```
Raw text -> 65-character tokenizer -> token windows -> token embeddings + sinusoidal positions
-> 6 causal Transformer blocks -> final LayerNorm -> vocabulary logits
-> cross-entropy next-character objective -> autoregressive generation
```

2. Transformer Foundations

2.1 Why Transformers? Sequential vs. Parallel Processing

A recurrent neural network processes a sequence through a chain of hidden states, so position t depends on position t-1. This creates T sequential operations for a sequence of length T. Self-attention instead compares all positions through matrix operations, reducing sequential depth and allowing modern GPUs to process many token interactions concurrently. The trade-off is memory: each attention head forms a T by T score matrix, so attention storage grows quadratically with context length.

Property	RNN	Self-Attention
Sequential operations	O(T)	O(1) layer depth
Compute per layer	O(T d^2)	O(T^2 d)
Pairwise-score memory	None	O(T^2) per head
Path between tokens	O(T)	O(1)

Table 1: Complexity Comparison

The repository includes an RNN-versus-attention experiment that measures sequence-processing time and attention memory. The experiment illustrates a practical nuance: a fused cuDNN recurrent layer can be very competitive at small widths, whereas an unfused educational attention implementation issues many small kernels. Nevertheless, the Transformer achieved the best and most stable validation loss in the end-to-end comparison, and its parallel-depth advantage becomes more important as sequence length, model width, and available hardware parallelism increase.

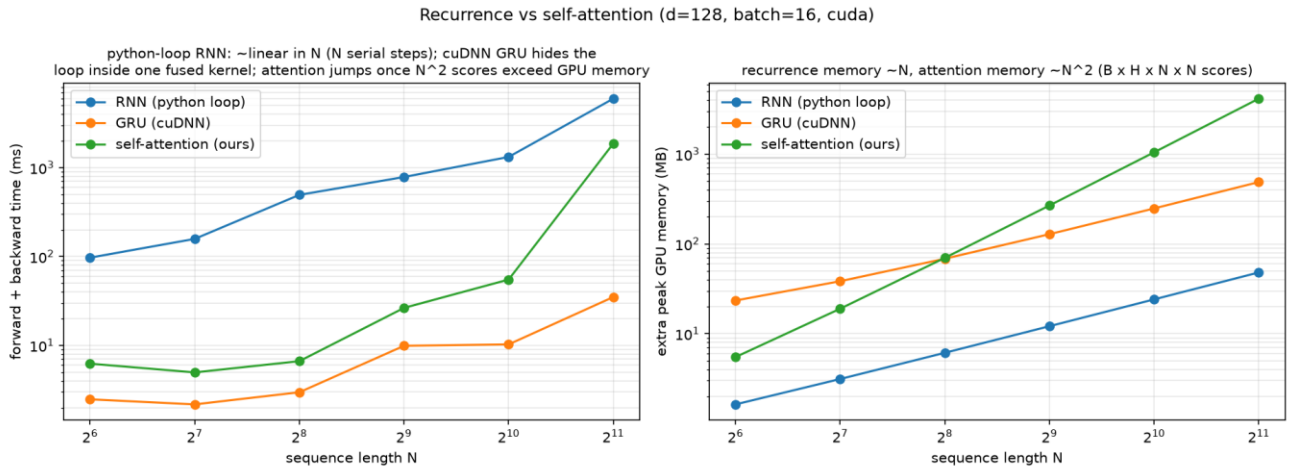


Figure 0: RNN versus self-attention speed and memory across sequence lengths.

2.2 Self-Attention: Queries, Keys, Values and Scaling

For input representations X, three learned projections create queries Q = XWq, keys K = XWk, and values V = XWv. A query represents what a position is looking for, a key represents what a position can match against, and a value represents the information that will be aggregated. The operation is:

Attention(Q,K,V) = softmax((QK^T / sqrt(d_k)) + M)V

The division by sqrt(d_k) prevents dot products from becoming excessively large as the head dimension increases. The additive mask M is zero for visible positions and negative infinity for future positions. Because masking occurs before softmax, blocked locations receive exactly zero probability while every valid attention row still sums to one.

2.3 Hand Calculation of Attention

A small three-token example is provided in the experiments to verify scaled dot-product attention independently of the full training pipeline. The calculation computes QK^T, applies scaling, evaluates row-wise softmax, multiplies by V, and then repeats the process using a causal mask. The script output is checked against an independent implementation so that a conceptual error cannot remain hidden inside the larger model.

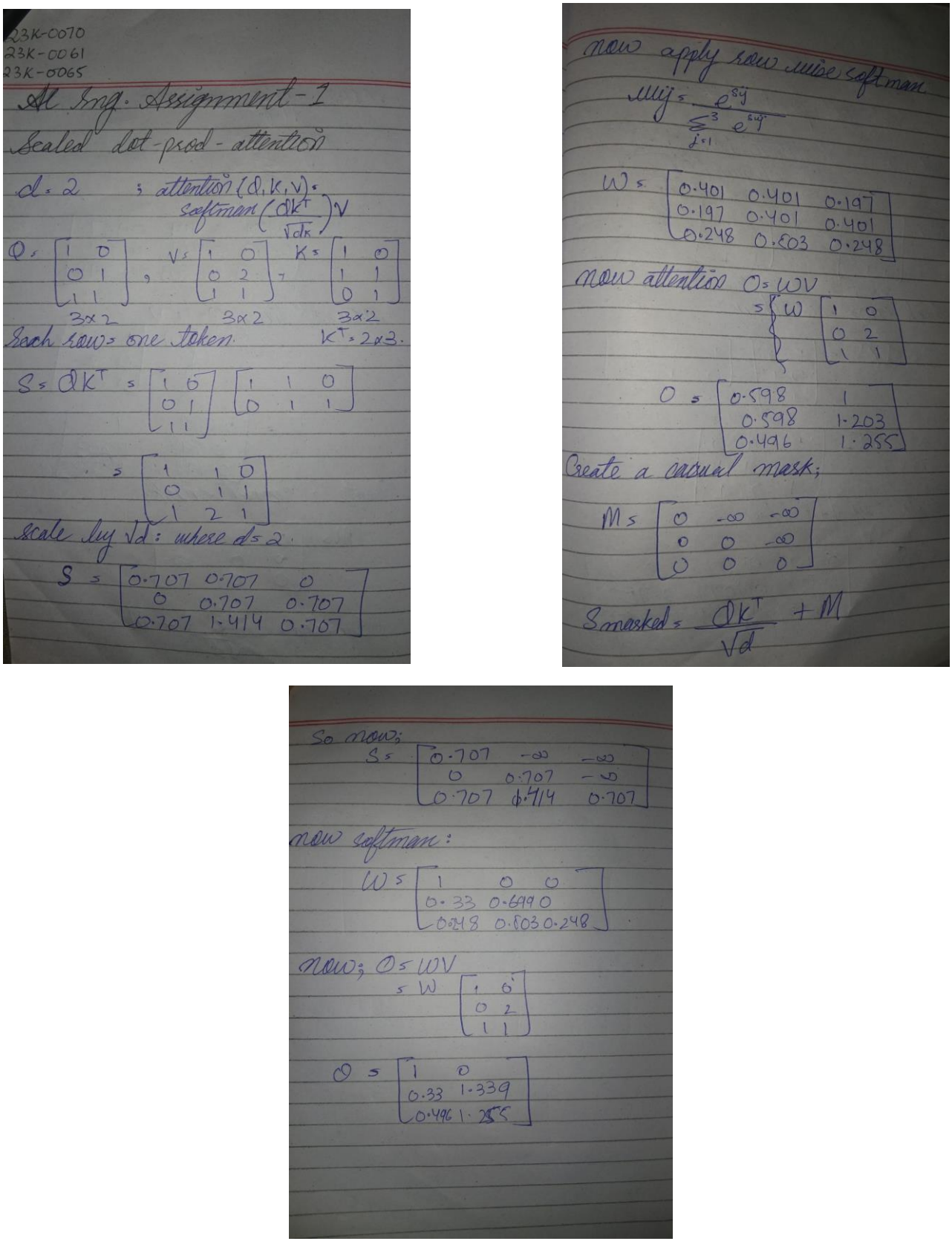


Figure 1.1(3 pages): Hand-calculated attention example showing QK^T, scaling, row-wise softmax and the Final weighted value vectors.

Figure 1 - hand-calculated attention (top: no mask, bottom: causal mask)

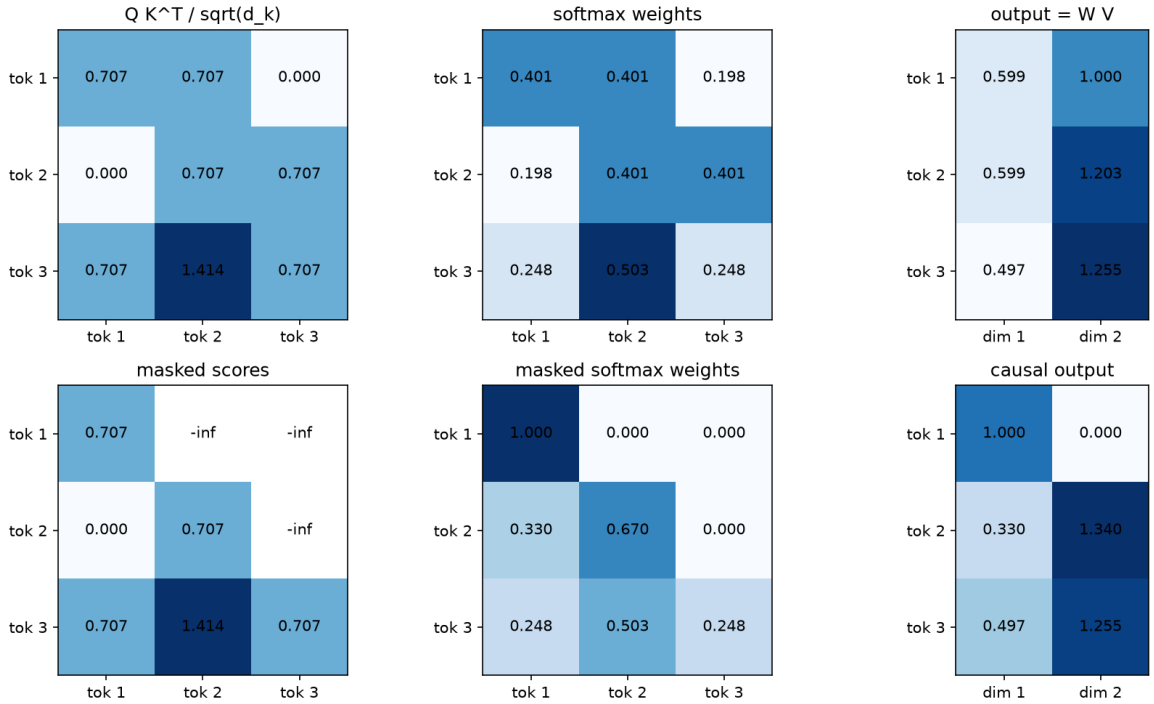


Figure 1.2: Calculated attention example showing QK^T, scaling, row-wise softmax and the final weighted value vectors using python.

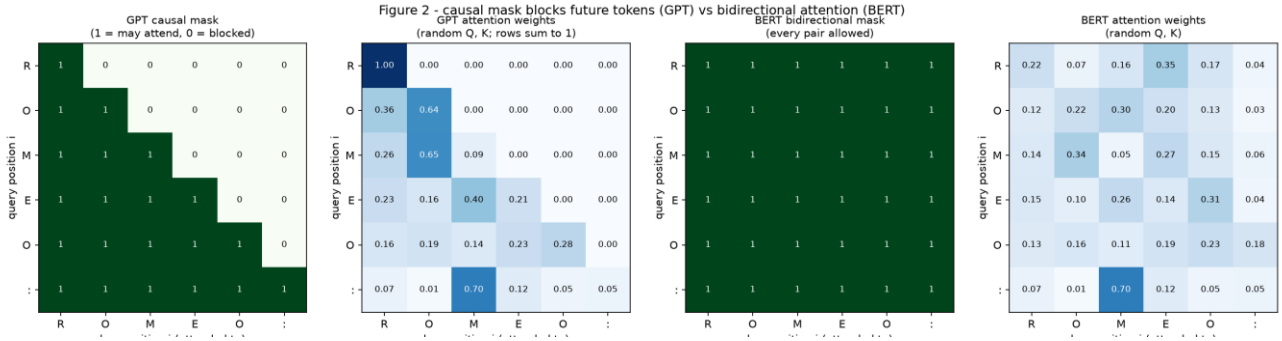


Figure 2: Causal and bidirectional attention masks, demonstrating that future positions receive zero attention probability.

2.4 Multi-Head Attention and Tensor Shapes

MiniGPT divides d_model = 256 into four heads of 64 dimensions. Each head can learn a different pattern while preserving the total representation width. The independent head outputs are concatenated and transformed by an output projection.

Stage	Shape	Operation
Input x	(B,T,256)	Token representations
Q, K, V	(B,T,256)	Three learned projections
Split heads	(B,4,T,64)	Reshape and transpose
Attention scores	(B,4,T,T)	One matrix per head
Per-head output	(B,4,T,64)	Weights multiplied by V
Merge heads	(B,T,256)	Transpose and concatenate
Output projection	(B,T,256)	Linear 256 to 256

Table 2: Multi-Head Attention Shape Trace

```
MULTI-HEAD ATTENTION SHAPE TRACE  (B=2, T=8, d_model=256, n_heads=4, d_head=64)
================================================================================

[A] full model forward pass (first block traced in detail)
  token ids                   (2, 8)  (B, T)
  token embedding * sqrt(d)   (2, 8, 256)  (B, T, d_model)
  positional encoding         (8, 256)  (T, d_model) broadcast over batch
  input x                     (2, 8, 256)  (B, T, d_model)
  Q = x W_q, K, V             (2, 8, 256)  (B, T, d_model)
  split_heads(Q)              (2, 4, 8, 64)  (B, H, T, d_head)
  scores / weights per head   (2, 4, 8, 8)  (B, H, T, T)
  weights @ V  (per head)     (2, 4, 8, 64)  (B, H, T, d_head)
  merge_heads (concat)        (2, 8, 256)  (B, T, d_model)
  output projection W_o       (2, 8, 256)  (B, T, d_model)
  block output (after FFN)    (2, 8, 256)  (B, T, d_model)
  final LayerNorm -> LM head  (2, 8, 65)  (B, T, vocab_size)
  attention maps returned: 6 layers x (2, 4, 8, 8)

[B] attention inside each head, then concatenation + output projection
  Q, K, V after split_heads: (2, 4, 8, 64)
  head 0: Q_i (2, 8, 64) -> weights (2, 8, 8) -> out (2, 8, 64)   (row sums = 1.000)
  head 1: Q_i (2, 8, 64) -> weights (2, 8, 8) -> out (2, 8, 64)   (row sums = 1.000)
  head 2: Q_i (2, 8, 64) -> weights (2, 8, 8) -> out (2, 8, 64)   (row sums = 1.000)
  head 3: Q_i (2, 8, 64) -> weights (2, 8, 8) -> out (2, 8, 64)   (row sums = 1.000)
  concat heads along features: 4 x (2, 8, 64) -> (2, 8, 256)
...
  W_o projection: (2, 8, 256) @ (256, 256) -> (2, 8, 256)
  equals MultiHeadAttention module output: True

saved outputs\figures\fig3_shape_trace.txt
```

Figure 3: Multi-head attention tensor-shape trace from input projection to merged output.

2.5 Positional Encoding

Self-attention alone is permutation-equivariant and therefore requires a positional signal. MiniGPT uses fixed sinusoidal positional encoding. Even dimensions use sine and odd dimensions use cosine at geometrically spaced frequencies. This introduces no trainable parameters and gives each context position a distinct pattern. Learned positions and no-position variants remain configurable for experiments.

$$PE(pos,2i) = sin(pos / 10000^{(2i/d_model)})$$
$$PE(pos,2i+1) = cos(pos / 10000^{(2i/d_model)})$$

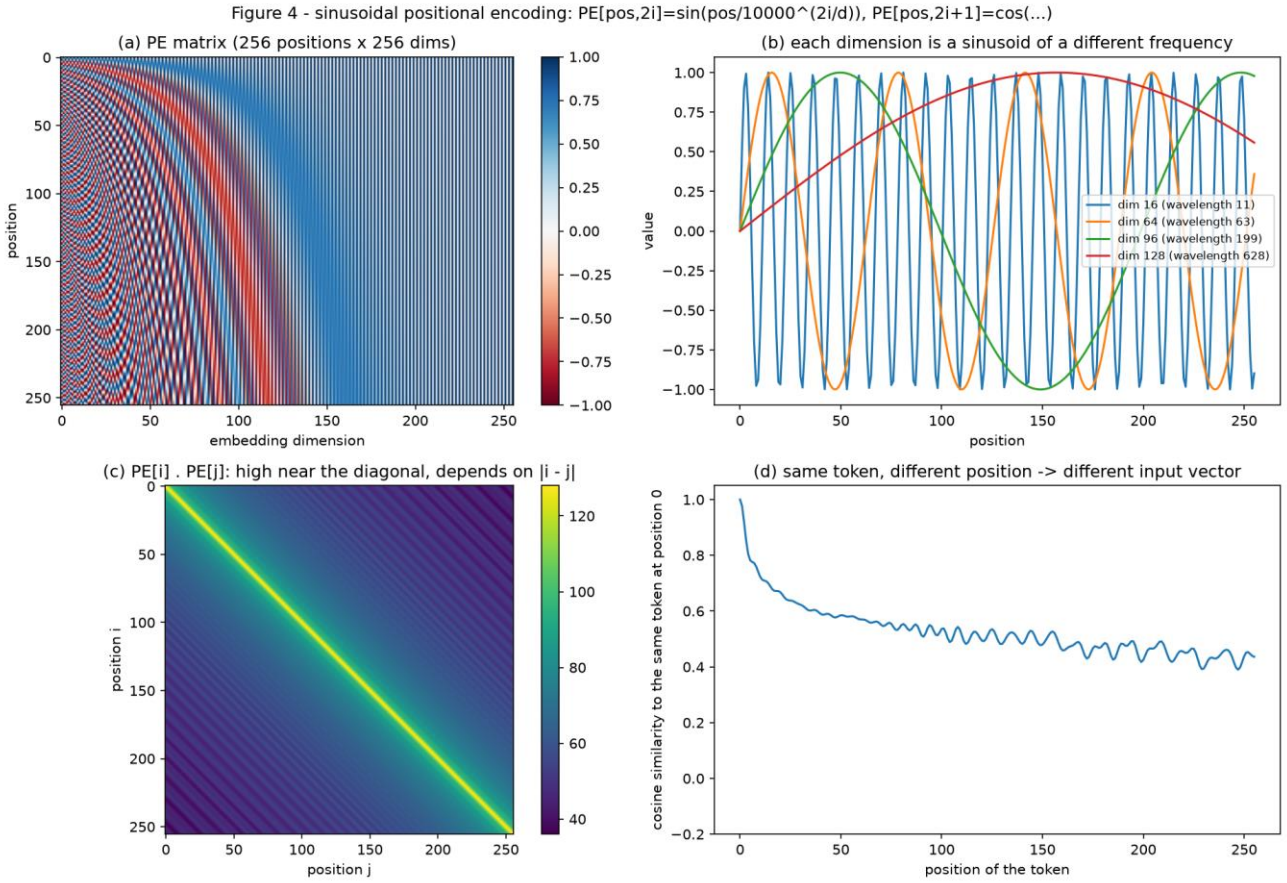


Figure 4: Sinusoidal positional encoding across sequence positions and embedding dimensions.

2.6 The Transformer Block

Each of the six decoder blocks follows a pre-LayerNorm design: x = x + MHA(LN(x)), followed by x = x + FFN(LN(x)). The feed-forward network expands 256 dimensions to 1024, applies GELU, and projects back to 256. Residual connections provide direct gradient paths, while pre-normalization stabilizes optimization without requiring a long warm-up.

The implementation scales token embeddings by sqrt(256) before adding positional encoding, preventing the fixed plus-or-minus-one sinusoidal signal from dominating small initialized embeddings. Residual output projections use GPT-2-style scaling by 1/sqrt(2 x number of layers), which controls residual-stream growth in deeper networks.

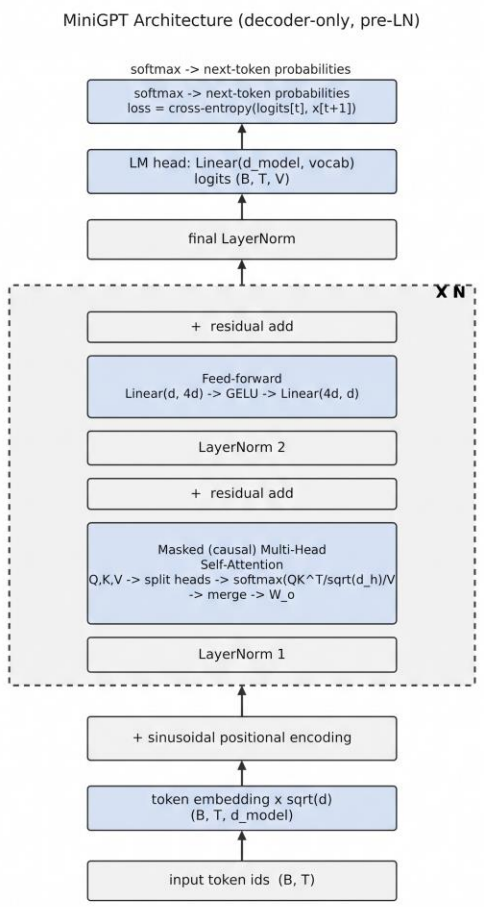


Figure 5: MiniGPT architecture from token IDs and positional encoding through six pre-normalized decoder blocks to vocabulary logits.

2.7 GPT Causal LM vs. BERT Masked LM

Property	BERT-style MLM	GPT-style CLM
Training target	Masked positions	Next token at every position
Attention	Bidirectional	Causal
Visible context	Left and right	Previous tokens only
Primary use	Representation learning	Left-to-right generation

Table 3: BERT vs GPT

The project includes a controlled comparison among causal language modeling, masked language modeling, and an intentionally leaky next-token model trained without a causal mask. The leaky model obtains an apparently excellent training loss because it can copy information from the future,

yet its honest next-token loss is poor and generation collapses. This experiment demonstrates that low loss is meaningful only when the information constraints match deployment.

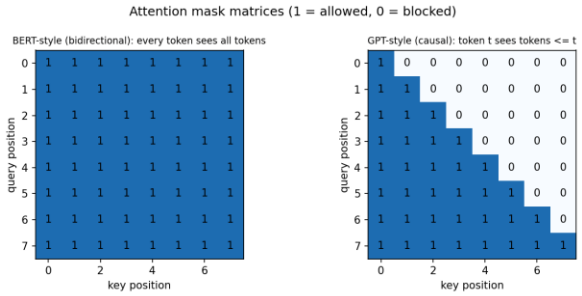


Figure 6: BERT-style bidirectional attention and GPT-style causal attention.

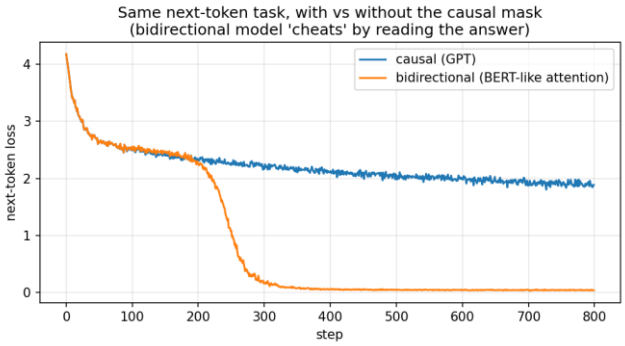


Figure 7: Honest causal LM, masked LM and leaky next-token training comparison.

3. Implementation

The system uses basic PyTorch building blocks rather than transformers, nn.Transformer, or nn.MultiheadAttention. This keeps every important operation visible and testable.

File or Directory	Purpose
src/attention.py	Causal masks, scaled dot-product attention, head split/merge and multi-head attention
src/positional_encoding.py	Sinusoidal and optional learned positional encodings
src/transformer_block.py	Pre-LN attention and feed-forward residual block
src/gpt.py	Complete decoder-only GPT model and loss
src/tokenizer.py	Deterministic 65-character encoder and decoder
src/dataset.py	Fixed-window next-character batches
src/train.py	Optimization, evaluation, logging, checkpointing and resume
src/generate.py	Command-line autoregressive inference

src/api.py	FastAPI web interface and REST endpoints
src/checkpoint.py	Portable checkpoint loading and validation
src/numpy_gpt.py	Lightweight NumPy inference for deployment
experiments/	Concept experiments, baselines, evaluation and ablations
tests/	Attention, masking, shape, pipeline, checkpoint, generation and API tests
outputs/	Logs, samples, metrics, figures and ablation results

Table 4: Implementation Breakdown

3.1 Tokenizer

The tokenizer is character-level and deterministic. Its vocabulary is the sorted set of unique characters in the training split, producing 65 symbols: newline, space, punctuation, uppercase letters and lowercase letters. No BOS, EOS or PAD tokens are required because the corpus is treated as one continuous stream and cut into fixed windows. Unsupported prompt characters are dropped and returned in ignored_chars instead of being mapped to an untrained unknown token.

This design keeps the embedding and output layers compact and makes tokenization transparent. Its disadvantage is that the model must learn spelling and word boundaries from individual characters, increasing sequence length and contributing to plausible but invented words.

3.2 Final Model Configuration

Parameter	Value	Reason
Vocabulary	65 characters	Determined by training split
Context length	256	Captures longer local structure than 64 or 128
d_model	256	Balanced capacity and 4 GB GPU memory
Attention heads	4	64 dimensions per head
Transformer layers	6	Best depth trend in ablations
FFN width	1024	Four times d_model
Dropout	0.2	Controls overfitting
Position encoding	Sinusoidal	Parameter-free and strongest tested option
Parameters	4,772,352	Compact educational model

Table 5: Final Model Configuration

3.3 Attention and Causal Masking in Code

```
scores = q @ k.transpose(-2, -1) / sqrt(head_dim)
scores = scores.masked_fill(~causal_mask, -inf)
weights = softmax(scores.float(), dim=-1).to(q.dtype)
output = weights @ v
```

Softmax is evaluated in float32 under mixed precision to improve numerical stability. The manual attention implementation can be replaced by PyTorch scaled-dot-product attention for an experiment, and tests verify that the fused and manual implementations are equivalent within tolerance.

3.4 Training Loop

- Sample 64 random windows of 256 characters from the training split.
- Shift each window by one position to form next-character targets.
- Produce logits of shape (64, 256, 65) and average cross-entropy over all positions.
- Use autocast with bfloat16 on the RTX 3050 while retaining numerically sensitive operations in float32.
- Backpropagate, clip gradient norm to 1.0 and update parameters with AdamW.
- Warm the learning rate for 200 steps, then follow cosine decay from 2e-3 toward 2e-4.
- Evaluate periodically, save best and last checkpoints, write JSONL metrics and generate qualitative samples.

4. Dataset and Preprocessing

Item	Value
Dataset	Tiny Shakespeare
Total size	1,115,394 characters; approximately 40,000 lines
Train split	1,003,854 characters (90%)
Validation split	55,770 characters (5%)
Test split	55,770 characters (5%)
Vocabulary	65 sorted unique training characters
Context	256 characters
Integrity	SHA-256 checked by preparation script
Special tokens	None

Table 6: Dataset and preprocessing

The preparation script downloads the same raw text used by the Tiny Shakespeare dataset, verifies its checksum, and applies a contiguous 90/5/5 character split. A contiguous split avoids overlap leakage that would occur if random windows from the same passages were independently assigned to training

and evaluation. It also exposes a real domain-shift issue: much of the test split comes from The Tempest, which the model did not see during training.

5. Training Setup

Setting	Final Run
Hardware	NVIDIA RTX 3050 Laptop GPU, 4 GB
Software	PyTorch 2.11, Python 3.12, Windows 11
Optimizer	AdamW; beta1 0.9, beta2 0.95
Peak learning rate	2e-3, selected from six-point sweep
Schedule	200-step linear warm-up, cosine decay to 2e-4
Weight decay	0.1 on matrices only
Batch	64 x 256 = 16,384 tokens per step
Steps	3,000; approximately 49 million processed tokens
Precision	bfloat16 autocast
Gradient clip	1.0
Regularization	Dropout 0.2 and best-validation checkpointing
Training time	531 s training; 643 s including evaluations
Throughput	92.6 thousand tokens/s
Peak GPU memory	1.8 GB
Best checkpoint	Step 2,750; validation loss 1.3531

Table 7: Training setup used

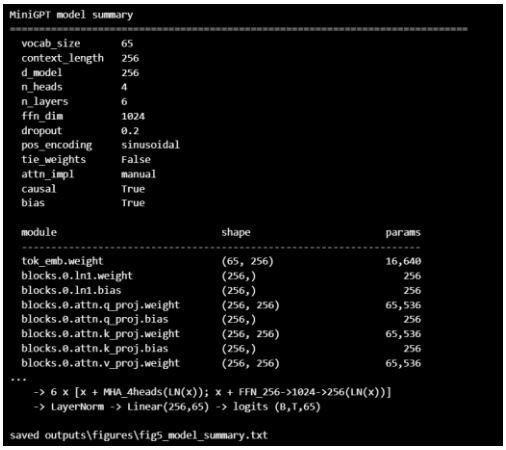


Figure 8: Model summary and parameter count

5.1 Debugging Gate: Overfitting a Tiny Sequence

Before full training, the model is required to memorize a small 65-character example. Loss decreases from approximately 4.2 to about 0.03. This verifies the interaction among target shifting, masking, gradient flow, optimizer behavior and checkpoint logic. A model that cannot pass this gate is not allowed to proceed to the expensive run.

5.2 Learning-Rate Selection and Stopping Point

Peak learning rates of 3e-4, 6e-4, 1e-3, 2e-3, 3e-3 and 4e-3 were evaluated for 1,500 steps. Improvements flattened beyond 2e-3, so 2e-3 was retained as a safer peak for the longer run. An earlier 5,000-step experiment achieved its best validation loss around step 3,000 and then overfit: training loss continued falling while validation loss rose. The final schedule was shortened to 3,000 steps so cosine decay ends near the observed validation minimum.

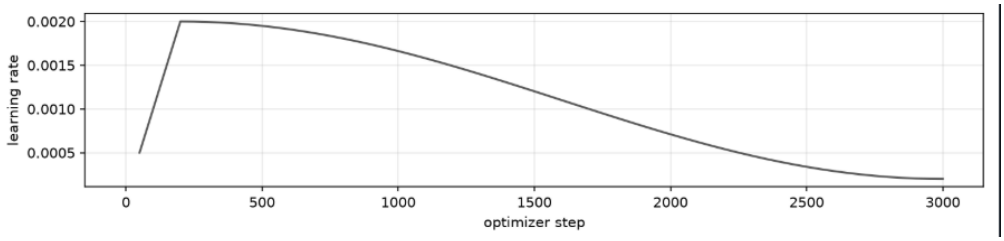


Figure 9: Learning-rate sweep and selected peak learning rate of 2e-3.

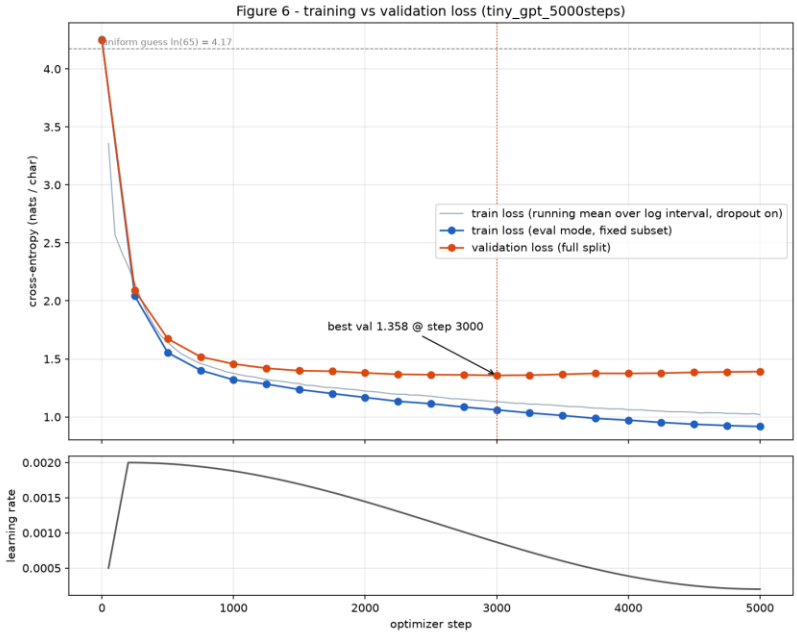


Figure 10: Five-thousand-step run showing validation overfitting after approximately step 3,000.

6. Results

6.1 Final Model Performance

Metric	Validation	Test
Cross-entropy loss	1.353 nats/char	1.530 nats/char
Perplexity	3.87	4.62
Top-1 next-character accuracy	60.0%	56.7%
Top-5 next-character accuracy	87.0%	85.8%
Bits per character	1.95	2.21

Table 9: Final Model performance

The final training loss was 1.065 and the final validation loss was 1.355; the released checkpoint is the step-2,750 model with validation loss 1.353. The train-validation gap shows moderate overfitting, which is expected for 4.77 million parameters trained repeatedly over approximately one million characters.

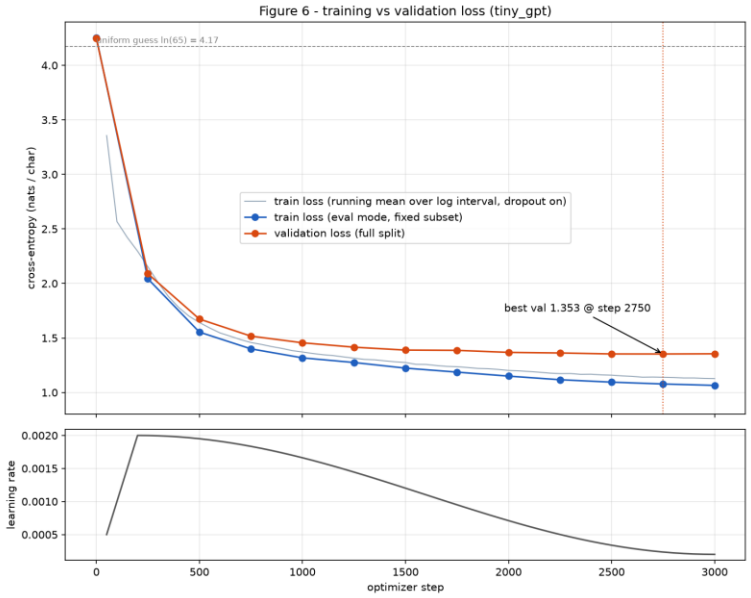


Figure 11: Training and validation loss curves for the final 3,000-step run.

6.2 Baseline Comparison

Model	Val Loss	Perplexity	Bits/Char
Uniform guess	4.174	65.0	6.02
Unigram	3.333	28.0	4.81
Bigram, add-one	2.474	11.9	3.57

Trigram, add-one	2.037	7.7	2.94
5-gram, add-one	2.123	8.4	3.06
2-layer LSTM, 4.81M	1.360	3.90	1.96
MiniGPT, 4.77M	1.353	3.87	1.95

Table 10: Loss comparison

MiniGPT slightly outperformed the parameter-matched LSTM and substantially outperformed all n-gram baselines. The LSTM reached its best value earlier but overfit more sharply, whereas MiniGPT remained comparatively stable near its best checkpoint.

6.3 Generated Sample

Prompt: ROMEO:
Generated: ROMEO:
As foul shadows not be great as to say,
As much as your mother like a common to death.
God grant you, to this uncle is commended:
I am not like a little fall for the prince,
Which is but unite as a d

The output reproduces speaker labels, line breaks, punctuation patterns and Shakespearean lexical style. It remains locally plausible but lacks global semantics and narrative consistency. Approximately 97% of generated words in the fixed evaluation are real words found in the corpus, while temperature 0.8 avoids repeated four-grams in the measured sample.

```
Prompt: ROMEO:
Generated: ROMEO:
Far is false; 'tis gone.

MERCUTIO:
Come, you sup your grace is that will conceive
Against me with contrives with the great England.

ROMEO:
That Hermione that we may have proved him within,
Whose canst will we are assured his daughter's mouth?

MERCUTIO:
The case, my lord, the head that send were

model parameters: 4.77M (4,772,352)
architecture: 6 layers, 4 heads, d_model 256, sinusoidal positions, vocab 65 (character-level)
context length: 256
temperature: 0.8
seed: 42
checkpoint: checkpoints\minigpt_shakespeare.pt (step 2750, val loss 1.3531)
device: cuda | 300 new tokens in 2.66s
```

```
KING HENRY:
They are found to hung their destinies.

WARWICK:
Now Warwick, I am the Duke of Lancaster.

KING RICHARD II:
Why is this, you will swear the field of his sword,
Not hear me, seeing in the king, and benefit sing
To lend my head this enemy, here the queen
Shall the noble Namer to Brittany call thee:
```

```
Prompt: To be, or not to be
Generated: To be, or not to be my priest honesty.
Tell me, if any more in thing I have drone,
Lest weep not be ashamed in what way is there
For revenge to be his heart that speaks me from thee
To say he shall be in honour of any h
```

Figure 12: Fixed-prompt generation samples at several temperatures and sampling strategies.

6.4 Attention Inspection

Trained attention maps remain strictly lower triangular. Early heads tend to focus on the current and immediately preceding characters, which is useful for spelling and punctuation. Deeper heads place more probability on positions 30 to 50 characters back and sometimes form vertical stripes around structural markers such as a line break after a speaker label. These visualizations demonstrate specialization, although a heatmap alone cannot prove a unique linguistic function for any particular head.

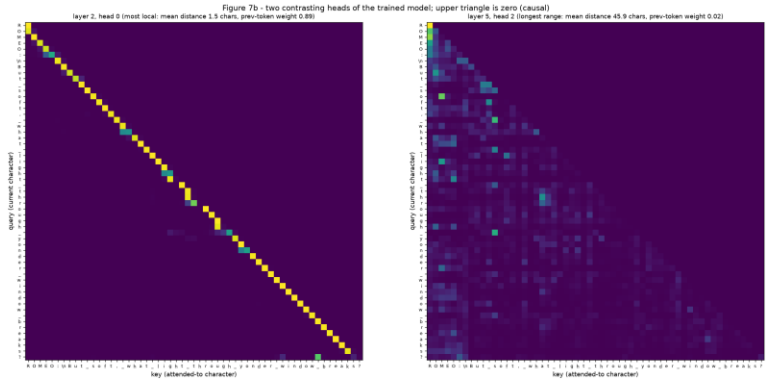


Figure 13: Detailed attention head view with token labels and attention-distance statistics.

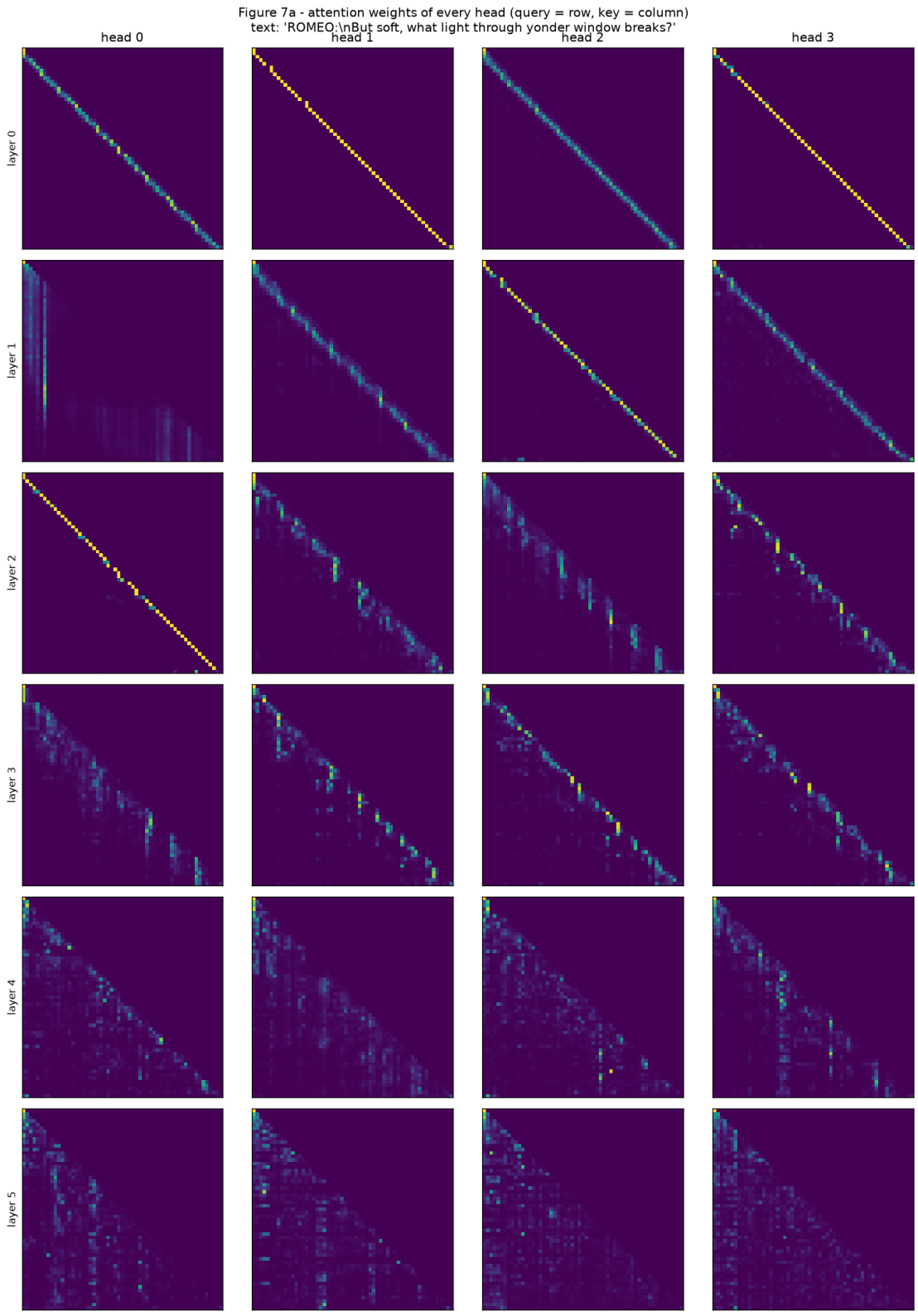


Figure 14: Attention weights for all heads and layers on a Shakespeare-style prompt.

7. Ablation Study

A smaller controlled model with d_model 128, four layers, four heads, context 128 and approximately 0.81 million parameters was used. Each factor changed independently, each setting used 3,000 steps and three random seeds, and results are reported as mean plus or minus standard deviation. Step time and peak memory come from a controlled benchmark.

Factor	Settings	Best Val Loss	Step Time	Peak Memory
Depth	2 / 4 / 6 layers	1.630 / 1.533 / 1.495	27 / 45 / 71 ms	185 / 331 / 476 MB
Heads	2 / 4 / 8	1.540 / 1.533 / 1.548	45 / 45 / 40 ms	275 / 331 / 472 MB
Context	64 / 128 / 256	1.529 / 1.533 / 1.573	50 / 45 / 45 ms	275 / 331 / 472 MB
Position	sinusoidal / learned / none	1.533 / 1.551 / 1.720	45 / 49 / 45 ms	331 MB

Table 11: Ablation results

7.1 Depth: 2 vs. 4 vs. 6 Layers

Validation loss improved consistently as depth increased, but step time and memory also rose almost linearly. The gain from two to four layers was larger than the gain from four to six, indicating diminishing returns. A plausible interpretation is that deeper layers compose more attention hops and convert short character features into broader text patterns.

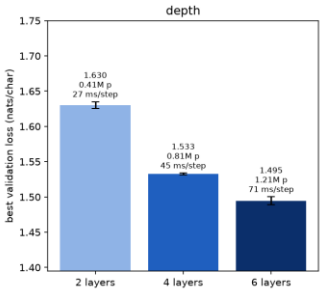


Figure 15.1: Depth ablation for 2, 4 and 6 layers.

7.2 Heads: 2 vs. 4 vs. 8

The three head settings were within about 0.015 nats. Eight heads were slightly worse because each head had only 16 dimensions when d_model was fixed at 128. This suggests that at small width, four 32-dimensional heads provide a better balance between per-head capacity and diversity. Memory grows with the number of heads because every head stores its own T by T map.

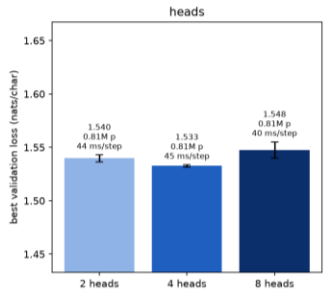


Figure 15.2: Attention-head ablation for 2, 4 and 8 heads.

7.3 Context Length: 64 vs. 128 vs. 256

Context 64 and 128 were effectively tied, while context 256 was worse under the fixed training budget. With equal tokens per step, longer windows mean fewer independent windows per batch and larger attention maps. The small model appears to learn mostly local character dependencies within 3,000 steps. This does not imply that long context is intrinsically harmful; it indicates that longer context needs sufficient model capacity and training to provide a return.

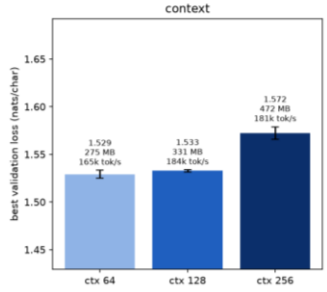


Figure 15.3: Context-length ablation showing validation loss, step time and peak memory.

7.4 Positional Encoding

Removing positional information increased validation loss by approximately 0.19 nats, the largest degradation among the controlled one-factor changes. Learned positions were slightly worse than sinusoidal positions. Without explicit positions, causal visibility still provides a weak order signal because later tokens have more predecessors, but content order remains difficult to represent.

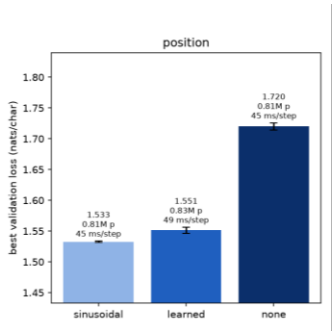


Figure 15.4: Sinusoidal, learned and no-position encoding ablation.

7.5 Model-Selection Candidates

Candidate	Parameters	Dropout	Best Val	Train Loss	Time
Final: d256, 6L, 4H	4.77M	0.2	1.353	1.065	9 min
More regularization	4.77M	0.3	1.363	1.166	8 min
Larger: d384, 6L, 6H	10.70M	0.3	1.360	1.104	14 min

All three candidates landed within 0.01 nats. The larger model did not improve generalization, while stronger dropout reduced the train-validation gap but learned more slowly. The evidence indicates that the project is data-limited: extra capacity mostly increases memorization, so the smaller and faster final model was retained.

7.6 Decoding Strategies

Greedy decoding has a high real-word rate but falls into repetitive loops, with approximately 35% repeated four-grams in the controlled sample. Temperatures from 0.5 to 0.8 produce fluent and non-repeating text. Temperature 1.3 increases diversity but also misspellings. Top-k and top-p filtering offer middle-ground strategies by removing very unlikely characters while preserving randomness.

4 prompts x 3 samples x 300 characters (greedy: 1 sample per prompt).

strategy	valid words	distinct-1	distinct-2	4-gram repetition	model NLL
greedy (T=0)	0.986	0.338	0.488	0.350	0.529
T=0.5	0.996	0.700	0.963	0.000	0.775
T=0.8	0.979	0.806	0.989	0.000	0.886
T=1.0	0.950	0.856	0.993	0.000	1.043
T=1.3	0.897	0.905	0.999	0.000	1.374
T=0.8, top-k=10	0.972	0.782	0.988	0.000	0.893
T=1.0, top-p=0.9	0.969	0.810	0.991	0.000	0.937

Figure 16: Greedy, temperature, top-k and top-p decoding comparison.

8. Failure Analysis

Failure 1: Local Fluency without Global Meaning

The model reproduces Shakespearean formatting and plausible words, but cannot maintain plot, character consistency or factual meaning across long generations. The 256-character sliding context, small corpus and character-level objective encourage local style modeling rather than durable semantic state. More diverse data, subword tokenization, larger context and larger capacity would be required for better coherence.

Failure 2: Overfitting on a One-Megabyte Corpus

The 5,000-step run reached training loss 0.917 while validation loss deteriorated to 1.391. Early stopping, dropout 0.2 and best-checkpoint selection reduce the impact, but the fundamental solution is more data rather than simply more model parameters.

Failure 3: Domain Shift in the Test Split

Overall test loss is 1.530, but it is 1.261 on text from a previously represented play and 1.693 on The Tempest, which was absent from training. This shows that a contiguous split gives a more demanding and informative test than random overlapping windows.

Failure 4: Greedy Repetition

Always choosing the most probable character causes deterministic loops. Temperature sampling of at least 0.5, optionally combined with top-k or nucleus filtering, is the practical mitigation.

Failure 5: Character Vocabulary Restrictions

The model knows only 65 symbols. Most digits, quotation marks, brackets, accented characters and emoji are removed from prompts and reported through ignored_chars. A byte-level or subword tokenizer would provide broader input coverage.

Failure 6: Fixed Context and No KV Cache

Generation uses a sliding 256-character window and forgets earlier content. In addition, each new character recomputes the complete active context because no key-value cache is implemented. This produces roughly 25 to 40 characters per second on a laptop CPU. A KV cache would avoid repeated attention projection and significantly improve autoregressive latency.

Failure 7: Experimental Uncertainty

Small differences should be interpreted relative to seed variance and experimental budget. The main ablations address this by using three seeds, but longer convergence studies and more hardware repetitions would strengthen runtime conclusions.

9. Engineering and Packaging

9.1 Automated Tests

The repository includes 59 tests covering attention math, causal masking, future-leakage controls, shape correctness, head splitting and merging, positional encoding, data preparation, tiny-sequence overfitting, checkpoint reload, generation reproducibility, NumPy/PyTorch agreement and API validation. This test suite is especially important because a Transformer can appear to train while containing subtle leakage or shape errors.

9.2 Command-Line Inference

```
python -m src.generate --prompt "ROMEO:" --max-new-tokens 200 --temperature 0.8 --seed 42
python -m src.generate --prompt "To be, or not to be" --top-k 10 --num-samples 3
python -m src.generate --prompt "JULIET:" --temperature 0
```

```
Prompt: ROMEO:
Generated: ROMEO:
Far is false; 'tis gone.

MERCUTIO:
Come, you sup your grace is that will conceive
Against me with contrives with the great England.

ROMEO:
That Hermione that we may have proved him within,
Whose canst will we are assured his daughter's mouth?

MERCUTIO:
The case, my lord, the head that send were

model parameters: 4.77M (4,772,352)
architecture: 6 layers, 4 heads, d_model 256, sinusoidal positions, vocab 65 (character-level)
context length: 256
temperature: 0.8
seed: 42
checkpoint: checkpoints\minigpt_shakespeare.pt (step 2750, val loss 1.3531)
device: cuda | 300 new tokens in 2.66s
```

Figure 17: Command-line inference output with prompt, architecture, checkpoint, speed and generated continuation.

9.3 REST API

Endpoint	Purpose
GET /	Minimal browser interface
GET /health	Liveness check
GET /model-info	Model configuration, parameters and training metrics
POST /generate	Validated text-generation request and response

POST /generate accepts a prompt, up to 1,000 new tokens, temperature from 0 to 2, optional top-k, top-p and seed fields. If MINIGPT_API_KEYS is configured, generation requires a matching X-API-Key header while health and model-information endpoints remain public.

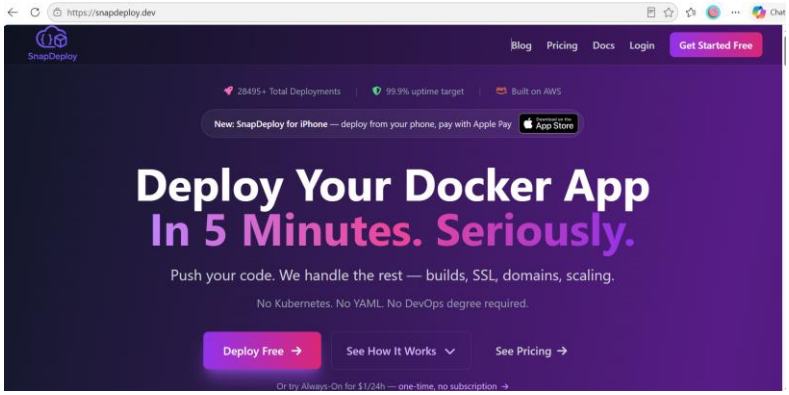


Figure 18.1: FastAPI Swagger documentation showing the /generate request schema and response.

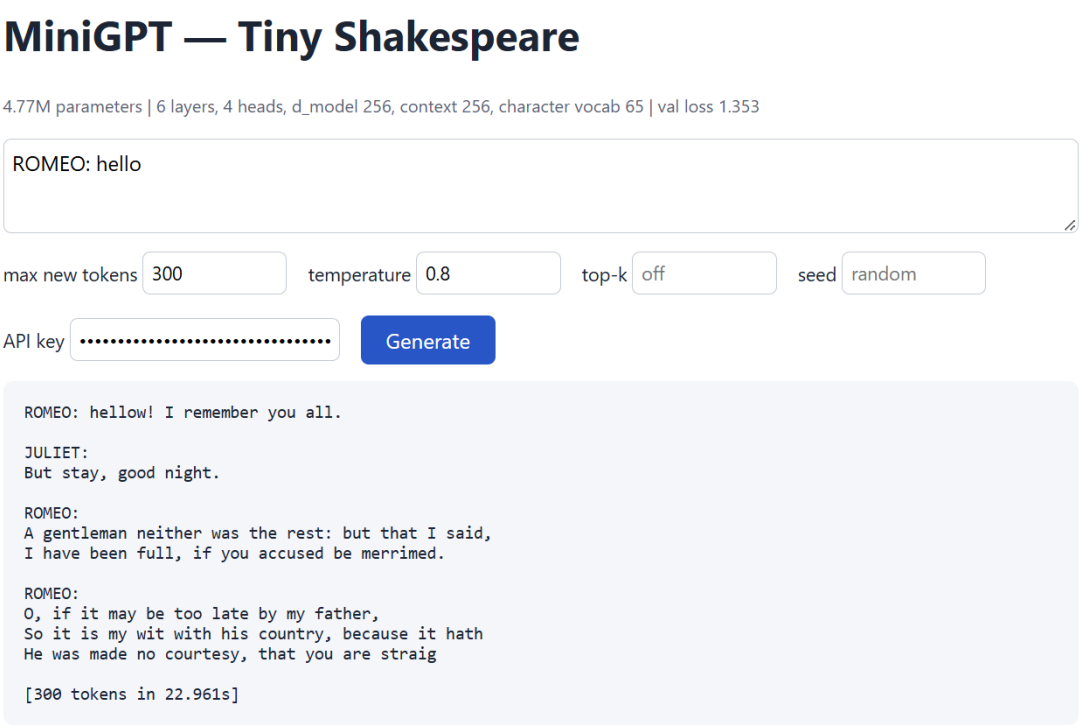


Figure 18.2: Live MiniGPT web interface and generated response.

9.4 Docker and Lightweight NumPy Serving

Training uses PyTorch, but the deployment path exports weights to an NPZ file and serves inference through a NumPy implementation verified against PyTorch logits and greedy output. This reduces deployment dependencies and supports container platforms that reject or struggle with heavyweight PyTorch images. The Docker build retrieves the released model artifact and starts the FastAPI service. Free hosting may sleep after inactivity, creating a 30- to 60-second cold start.

```
docker build -t minigpt .
docker run -p 8000:8000 minigpt
```

[IMAGE TO BE INSERTED]

Figure 11d: Docker build and deployed container health/model-info responses.

9.5 Reproducing the Project

```
git clone https://github.com/AbdulRahmanAzam/minigpt-shakespeare.git
cd minigpt-shakespeare
python -m venv .venv
pip install -r requirements.txt
python scripts/prepare_data.py
python -m pytest
python -m src.train --config configs/overfit.yaml
python -m src.train --config configs/tiny_gpt.yaml
bash scripts/reproduce.sh
```

The Colab notebook contains saved outputs from a complete run. The build_notebook script can regenerate and execute it. Each training run writes the resolved configuration, JSONL metrics, generated samples, a summary and best/last checkpoints, enabling direct audit of reported results.

10. Conclusion

This project asked whether a compact decoder-only Transformer built from first principles could learn the next-character structure of a narrow text domain. The answer is yes for style, syntax and short-range structure, but only partially for meaning. MiniGPT reduced validation loss to 1.353 nats per character, achieved perplexity 3.87 and predicted the correct next character 60.0% of the time, while using only 4.77 million parameters and approximately nine minutes of laptop-GPU training.

- Attention provides parallel sequence processing but creates quadratic context memory.
- Causal masking is essential; without it, next-token training can leak the answer from future positions.
- Depth produced the clearest ablation improvement, while too many narrow heads offered no benefit at small width.
- Sinusoidal positional encoding was stronger than learned or absent positions under the available data budget.
- The final model is data-limited: a 10.7M-parameter candidate did not outperform the 4.77M model.
- Reproducibility requires more than a checkpoint. The project packages tests, configs, logs, experiments, CLI inference, an API, Docker deployment and acceptance checks.

The most valuable future improvements are a larger and more diverse corpus, subword or byte-level tokenization, rotary positions, a key-value cache, optimized attention kernels, longer-context training, quantization and repeated multi-seed scaling studies.s

11. References

a) Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS.
b) Radford, A. et al. (2018). Improving Language Understanding by Generative Pre-Training.
c) Radford, A. et al. (2019). Language Models are Unsupervised Multitask Learners.
d) Devlin, J. et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL.
e) Xiong, R. et al. (2020). On Layer Normalization in the Transformer Architecture.
f) Loshchilov, I. and Hutter, F. (2019). Decoupled Weight Decay Regularization. ICLR.
g) Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer Normalization.
h) Hendrycks, D. and Gimpel, K. (2016). Gaussian Error Linear Units.
i) Karpathy, A. char-rnn, nanoGPT and Tiny Shakespeare resources.
j) PyTorch documentation for automatic mixed precision and scaled dot-product attention.
k) MiniGPT project repository: https://github.com/AbdulRahmanAzam/minigpt-shakespeare